

Kubernetes Pod Affinity: preferredDuringSchedulingIgnoredDuringExecution

A portable manifest with multiple weights and a clear explanation

Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3

  selector:
    matchLabels:
      app: web

  template:
    metadata:
      labels:
        app: web

    spec:
      affinity:
        podAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:

            # Preference 1 – strongest
            # Prefer a node that already runs an API pod.
            - weight: 70
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app: api
                topologyKey: kubernetes.io/hostname

            # Preference 2
            # Also prefer a node that already runs a cache pod.
            - weight: 20
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app: cache
                topologyKey: kubernetes.io/hostname

            # Preference 3 – weakest
            # Also prefer a node that already runs a worker pod.
            - weight: 10
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app: worker
                topologyKey: kubernetes.io/hostname

      containers:
        - name: web
          image: nginx:1.27
          ports:
            - containerPort: 80
```

What preferredDuringSchedulingIgnoredDuringExecution means

preferredDuringScheduling

The rule is a **preference**, not a requirement. When Kubernetes is deciding where to place the web Pod, it looks at the available nodes and gives a higher scheduling score to nodes that satisfy these preferences. If no node satisfies a preference, Kubernetes can still schedule the Pod somewhere else.

IgnoredDuringExecution

Once the Pod has been successfully scheduled and is running, Kubernetes does not continuously enforce the affinity rule. For example: web-1 is scheduled onto node-a; node-a contains an API Pod, so the 70-weight preference helped select it; later, the API Pod is deleted; web-1 is not evicted simply because the affinity preference is no longer satisfied.

How multiple weights work

The three preferences have weights: API pod = 70, Cache pod = 20, Worker pod = 10.

Node	API	Cache	Worker	Preference score
node-a	✓	✓	✓	100
node-b	✓	✓	✗	90
node-c	✓	✗	✗	70
node-d	✗	✓	✓	30
node-e	✗	✗	✗	0

The scheduler can therefore rank them approximately:

node-a → 70 + 20 + 10 = 100

node-b → 70 + 20 = 90

node-c → 70 = 70

node-d → 20 + 10 = 30

node-e → 0 = 0

So **node-a is the most preferred**, but this is still a preference. A score of 0 does not automatically make a node invalid.

Why this is portable

The example deliberately uses `topologyKey: kubernetes.io/hostname` rather than depending on cloud-specific topology such as `topology.kubernetes.io/zone`. `kubernetes.io/hostname` is a standard Kubernetes node label and generally makes the example easier to move between cloud providers, on-premises clusters, and local Kubernetes environments.

However, **portability does not mean identical behavior everywhere**. The actual hostname values and the set of Pods running on each node are cluster-specific, so the resulting scheduling decision can differ between clusters.

One important limitation

This configuration does **not** guarantee placement. If you need: *“The Pod must run on a node satisfying this affinity.”* use `requiredDuringSchedulingIgnoredDuringExecution`.

preferredDuringSchedulingIgnoredDuringExecution

-
- Scheduling → preference
 - ■■■ weights influence the score
-
- Execution → ignored
 - Pod is not evicted when conditions change

In short: weights influence where Kubernetes would like to place the Pod; they do not create hard placement requirements, and the preference is not enforced after scheduling.